

Rapport — Labo 07 : architecture event-driven, event sourcing et pub/sub

LOG430-02 — Architecture logicielle, École de technologie supérieure (ÉTS)

Chargé de laboratoire : Gabriel C. Ullmann

Étudiant : Ralph Christian Gabriel

Code permanent : GABR77340401

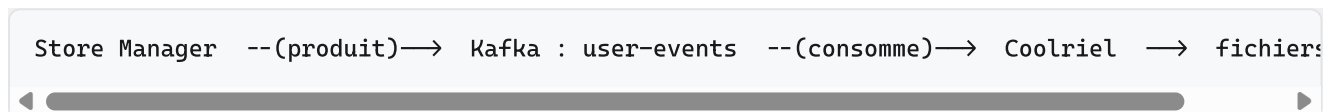
Session : Été 2026

Application : Store Manager (suite du Labo 03)

Date des mesures : 2026-06-10

Mise en contexte

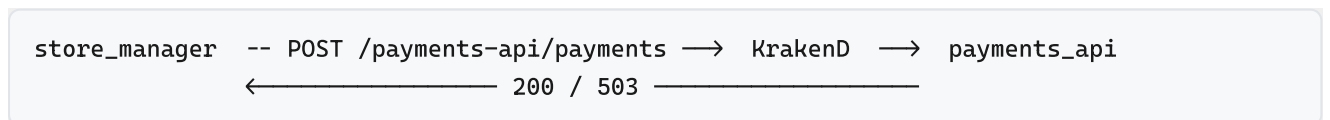
Dans ce labo, le Store Manager (labo 5) envoie des événements à Kafka quand un utilisateur est créé ou supprimé. Le microservice Coolriel écoute ces événements sur le topic `user-events` et génère un courriel HTML pour chacun, sans jamais appeler le Store Manager directement. Le producteur ne connaît donc pas le consommateur : il connaît seulement le nom du topic.



J'ai validé toute la chaîne en créant et en supprimant des utilisateurs via la collection Postman, puis en regardant les logs des deux conteneurs. Les extraits se trouvent en annexe.

Question 1 — Différence avec la communication `store_manager` ↔ `payments_api` du labo 5

Au labo 5, le Store Manager parle à `payments_api` avec une requête HTTP REST passant par la passerelle KrakenD. C'est synchrone : l'appelant connaît l'adresse du service, envoie la requête et attend la réponse avant de continuer.



Au labo 7, c'est l'inverse. Le Store Manager publie un événement sur Kafka et passe à la suite sans attendre. Il ne sait pas qui va le lire.

```
# store_manager - src/orders/commands/write_user.py (producteur)
UserEventProducer().get_instance().send('user-events', value={
    'event': 'UserCreated',
    'id': new_user.id, 'name': new_user.name, 'email': new_user.email,
    'user_type_id': new_user.user_type_id,
    'datetime': str(datetime.datetime.now())})
```

```
# coolriel - src/consumers/user_event_consumer.py (consommateur)
self.consumer = KafkaConsumer('user-events', bootstrap_servers=... , group_id=... ,
                               value_deserializer=lambda m: json.loads(m.decode('utf-8'))))
for message in self.consumer:
    self._process_message(message.value)    # génère le HTML
```

La conséquence pratique : avec REST, si `payments_api` est éteint, l'appel échoue tout de suite, et l'appelant doit gérer cette erreur. Avec Kafka, la réponse de `POST /users` revient même si Coolriel est arrêté; l'événement reste dans le topic et sera traité quand Coolriel reviendra.

Les deux approches ont leur place. Le REST synchrone est bon quand on a besoin de la réponse maintenant, par exemple confirmer un paiement avant de finaliser une commande. Le pub/sub Kafka est meilleur pour les effets de bord qui ne doivent pas bloquer le client, comme l'envoi d'un courriel. Le tableau résume les compromis que j'ai observés.

Critère	REST synchrone (labo 5)	Kafka pub/sub (labo 7)
Couplage	Fort : URL + attente	Faible : seulement le topic
Si le service cible tombe	L'appel échoue	L'événement est rejoué plus tard
Latence vue par le client	Inclut le service appelé	Quasi nulle
Cohérence	Forte (on a la réponse)	Éventuelle
Ajouter un consommateur (SMS, stats)	Modifier l'appelant	S'abonner au topic, rien à changer
Historique des événements	Aucun par défaut	Possible (rétention Kafka)

Le principal inconvénient du modèle Kafka, c'est qu'il est plus difficile à déboguer : il n'y a plus un appel et une réponse à suivre, mais un flux d'événements qui se traite ailleurs et plus tard.

Question 2 — Méthodes modifiées dans `write_user.py`

J'ai touché aux deux méthodes du fichier, `add_user` et `delete_user`.

Pour `add_user`, j'ai ajouté le paramètre `user_type_id`, je le sauvegarde en base, et je l'inclus dans l'événement `UserCreated` pour que Coolriel puisse personnaliser le message.

```
def add_user(name: str, email: str, user_type_id: int = 1):
    ...
    new_user = User(name=name, email=email, user_type_id=user_type_id)
    session.add(new_user); session.flush(); session.commit()
    UserEventProducer().get_instance().send('user-events', value={
        'event': 'UserCreated', 'id': new_user.id, 'name': new_user.name,
        'email': new_user.email, 'user_type_id': new_user.user_type_id,
        'datetime': str(datetime.datetime.now())})
    return new_user.id
```

Pour `delete_user`, le piège est que la suppression efface le nom, le courriel et le type. Si j'envoie l'événement après le `delete`, je n'ai plus rien à mettre dans le courriel d'au revoir. Je copie donc les données de l'utilisateur avant de le supprimer, puis j'émet `UserDeleted`.

```
def delete_user(user_id: int):
    ...
    user = session.query(User).filter(User.id == user_id).first()
    if user:
        deleted_user = {'id': user.id, 'name': user.name,
                        'email': user.email, 'user_type_id': user.user_type_id}
        session.delete(user); session.commit()
        UserEventProducer().get_instance().send('user-events', value={
            'event': 'UserDeleted', **deleted_user,
            'datetime': str(datetime.datetime.now())})
        return 1
    return 0
```

J'ai aussi fait quelques changements connexes : `user_controller.create_user` lit `user_type_id` dans le corps de la requête, le modèle `User` reçoit la colonne, et `db-init/init.sql` crée la table `user_types` avec la clé étrangère.

Question 3 — Vérification du type d'utilisateur

Le type voyage dans l'événement, dans le champ `user_type_id`. Côté Coolriel, chaque handler traduit cet identifiant en nom, puis choisit le bon gabarit. Si le type est inconnu (par exemple un vieil événement sans `user_type_id`), je retombe sur le gabarit client.

```
# handlers/user_created_handler.py (même logique dans user_deleted_handler.py)
USER_TYPES = {1: "client", 2: "employee", 3: "manager"}

def _resolve_template(self, user_type_id: int) → Path:
    templates_dir = Path(__file__).parent.parent / "templates"
    type_name = USER_TYPES.get(user_type_id, "client")
    candidate = templates_dir / f"welcome_{type_name}_template.html"
    if not candidate.exists():
        candidate = templates_dir / "welcome_client_template.html"
    return candidate

def handle(self, event_data):
    user_type_id = event_data.get('user_type_id', 1) # défaut : client
    template_path = self._resolve_template(user_type_id)
    ...
```

Résultat : un employé (type 2) reçoit « Salut et bienvenue dans l'équipe ! », un directeur (type 3) reçoit le message de la direction, et un client (type 1) garde le message d'accueil habituel. Le courriel d'au revoir suit la même règle avec les gabarits `goodbye_{type}`. Dans les logs, on voit `type=client`, `type=employee` ou `type=manager` selon le cas.

J'ai préféré une table de correspondance plus un choix de fichier plutôt qu'une cascade de `if`, parce que ça garde la logique de vérification au même endroit et qu'ajouter un quatrième type revient à ajouter une ligne et un gabarit.

Question 4 — Le partitionnement de Kafka et la performance en lecture

Voici les points que je retiens de la documentation de Kafka (sections Persistence et Efficiency).

Chaque partition est un journal en append-only : on écrit et on lit les messages de façon séquentielle. Les disques sont beaucoup plus rapides en accès séquentiel qu'en accès aléatoire, donc même sur un disque dur la lecture reste rapide. Kafka s'appuie aussi sur le cache de pages du système d'exploitation au lieu d'un cache en mémoire JVM, ce qui évite le double cache et la pression sur le ramasse-miettes.

Pour le débit, l'idée centrale est le partitionnement. Un topic est découpé en plusieurs partitions réparties sur les brokers, et on peut lire et écrire en parallèle sur ces partitions. Dans un groupe de consommateurs, Kafka distribue les partitions entre les consommateurs (au plus un consommateur par partition), donc on augmente la vitesse de lecture en ajoutant des consommateurs jusqu'au nombre de partitions.

Deux autres détails aident. Le suivi de la position de lecture (l'offset) est géré par le consommateur, pas par le broker, ce qui garde le broker léger et permet de rejouer l'historique, exactement ce dont l'event sourcing a besoin. Et pour livrer les messages, Kafka copie directement du cache de pages vers la socket réseau (zero-copy avec `sendfile`), ce qui évite des copies mémoire inutiles. Les messages sont aussi regroupés en lots et compressés, ce qui réduit le nombre d'opérations réseau.

En clair, Kafka va vite en lecture parce qu'il combine des journaux séquentiels servis depuis le cache de l'OS et un partitionnement qui se parallélise avec le nombre de consommateurs.

Question 5 — Nombre d'événements récupérés par le consommateur historique

Le `UserEventHistoryConsumer` lit depuis le début du topic (`auto_offset_reset="earliest"`), avec un `group_id` à part (`coolriel-group-history`) pour ne pas partager les partitions avec le consommateur live. Le paramètre `consumer_timeout_ms=5000` fait sortir la boucle cinq secondes après le dernier message. Comme l'énoncé le demande, je n'écris pas à chaque tour de boucle : j'accumule les événements et j'écris le fichier JSON une seule fois à la fin.

Lors de mon premier test manuel (trois créations et trois suppressions), le consommateur a récupéré 6 événements. Après le test de charge Locust, il en a récupéré 1103, ce qui confirme que la rétention de Kafka garde bien l'historique.

Extrait de `output/user_event_history.json` :

```
[
  { "event": "UserCreated", "id": 6, "name": "Grace Hopper",
    "email": "ghopper@example.com", "user_type_id": 1,
    "datetime": "2026-06-11 21:29:23.509258" },
  { "event": "UserCreated", "id": 7, "name": "New Employee",
    "email": "newemp@magasinducoin.ca", "user_type_id": 2,
    "datetime": "2026-06-11 21:29:23.740381" },
  { "event": "UserDeleted", "id": 1, "name": "Ada Lovelace",
    "email": "alovelace@example.com", "user_type_id": 1,
    "datetime": "2026-06-11 21:29:23.972067" }
]
```

```
UserEventHistoryConsumer - INFO - 1103 événement(s) historique(s) enregistré(s) dans output,
```

Activité 8 — Test de charge Locust

J'ai visé `store_manager:5000` directement, parce que la passerelle KrakenD n'expose pas la route de suppression d'utilisateur. Le scénario fait deux choses : dans 75 % des cas il crée puis supprime un utilisateur (ce qui déclenche `UserCreated` et `UserDeleted`), et dans 25 % des cas il crée seulement. Chaque utilisateur a un courriel unique, sinon la contrainte `UNIQUE` sur la colonne `email` rejette l'insertion. J'ai lancé 30 utilisateurs simultanés, démarrés à 5 par seconde, pendant 45 secondes.

Endpoint	Requêtes	Échecs	Moy (ms)	Méd (ms)	p95 (ms)	Max (ms)
POST /users	474	0	77	68	150	272
POST /users (création seule)	147	0	83	71	170	260
DELETE /users/[id]	474	0	70	61	130	230
Agrégé	1095	0 (0 %)	75	65	150	272

Débit d'environ 24,5 requêtes par seconde, et aucun échec.

Ce que je retiens : la production Kafka est asynchrone, donc `send()` ne bloque pas la réponse HTTP. C'est probablement pourquoi je n'ai eu aucun échec et des latences stables (médiane 65 ms, p95 150 ms) même si chaque requête émet un événement. Le vrai goulot reste MySQL, à cause des INSERT et DELETE et de la contrainte d'unicité, pas Kafka. Le `KafkaProducer` est un singleton, donc une seule connexion est réutilisée pour toutes les requêtes au lieu d'en rouvrir une à chaque fois. Enfin, tous les événements produits pendant le test se sont retrouvés dans Kafka et ont été relus ensuite par le consommateur historique, ce qui donne le total de 1103.

Annexe — Validation (logs de Coolriel)

```
UserEventConsumer - DEBUG - Evenement : UserCreated
Handler - DEBUG - Courriel HTML (type=client) genere a Grace Hopper (ID: 6), output/welco
Handler - DEBUG - Courriel HTML (type=employee) genere a New Employee (ID: 7), output/welco
Handler - DEBUG - Courriel HTML (type=manager) genere a New Boss (ID: 8), output/welco
Handler - DEBUG - Courriel d'au revoir (type=client) genere a Ada Lovelace (ID: 1), output
Handler - DEBUG - Courriel d'au revoir (type=employee) genere a Jane Doe (ID: 4), output
Handler - DEBUG - Courriel d'au revoir (type=manager) genere a Da Boss (ID: 5), output
```